📱 DevCore UAT - Documentación del Proyecto

📋 Información General

Nombre del Proyecto: DevCore - Plataforma de Servicios Estudiantiles UAT
Versión: 1.0.0
Fecha: Marzo 2026
Stack Tecnológico:

- Frontend: Kotlin + Jetpack Compose (Android) / HTML + CSS (Prototipo Web)
- Backend: Python (Pendiente implementación)
- Base de Datos: PostgreSQL/MongoDB (Pendiente)

🎯 Objetivo del Proyecto

Desarrollar una plataforma integral para estudiantes de la Universidad Autónoma de Tamaulipas (UAT) que integra:

- Sistema de registro y autenticación
- Monitoreo de transporte universitario en tiempo real
- Marketplace estudiantil
- Sistema de reputación y confianza
- Gamificación con streaks y logros

🎨 Paleta de Colores

Colores UAT (Paleta Pastel):

- **Naranja Pastel:** `#FFB39C` - Color principal, botones de acción
- **Azul Pastel:** `#8AB4D7` - Color secundario, fondos, gradientes
- **Gris Pastel:** `#B8BCC2` - Color terciario, textos secundarios, bordes
- **Fondo Claro:** `#FAFBFC` - Fondo principal de la aplicación
- **Blanco:** `#FFFFFF` - Cards, superficies elevadas

📱 Pantallas del Proyecto

1️⃣ Pantalla de Bienvenida (Splash Screen)

Archivo: `WelcomeScreen.kt` / `index.html`

Descripción: Primera pantalla que ven los usuarios al abrir la aplicación. Presenta el branding de DevCore y opciones de acceso.

Elementos Visuales:

- Fondo con gradiente vertical (Azul Pastel → Naranja Pastel)
- Logo de DevCore en el centro
- Título "DevCore UAT" con tipografía bold
- Subtítulo "Tu plataforma estudiantil de confianza"
- Dos botones de acción principales

Funcionalidades:

- ☑ Botón "Iniciar Sesión" - Redirige a pantalla de login
- ☑ Botón "Registrarse" - Redirige a pantalla de registro
- ☑ Animación de gradiente (implementada en versión Kotlin)

Estado de Implementación:

- ☑ Diseño visual completo
- ☑ Navegación funcional
- ⏳ Animaciones de entrada (pendiente)

---

2 Pantalla de Inicio de Sesión (Login)

Archivo: `LoginScreen.kt` / `login.html`

Descripción: Permite a usuarios registrados acceder a la plataforma mediante credenciales institucionales.

Elementos Visuales:

- Header con gradiente UAT
- Logo DevCore
- Formulario de login con campos estilizados
- Validación visual de campos
- Enlace a registro

Funcionalidades:

- ☑ Campo "Correo institucional" (validación `@uat.edu.mx`)
- ☑ Campo "Contraseña" con ocultamiento
- ☑ Botón "Iniciar Sesión"
- ☑ Enlace "¿No tienes cuenta? Regístrate"
- ⏳ Validación de credenciales (backend pendiente)
- ⏳ Recuperación de contraseña (pendiente)
- ⏳ Inicio de sesión con OAuth (pendiente)

Validaciones Planificadas:

- Verificar formato de email institucional
- Contraseña mínimo 8 caracteres
- Manejo de errores (credenciales incorrectas)
- Bloqueo por múltiples intentos fallidos

Estado de Implementación:

- ☑ UI completa
- ☑ Navegación funcional
- ⏳ Lógica de autenticación (requiere backend)

---

3 Pantalla de Registro

Archivo: `RegisterScreen.kt` / `register.html`

Descripción: Permite a nuevos usuarios crear una cuenta en la plataforma con su correo institucional.

Elementos Visuales:

- Header con gradiente
- Formulario extenso con múltiples campos
- Selector de archivo para ficha de pago
- Indicadores de validación en tiempo real

Funcionalidades:

- ☑ Campo "Nombre completo"
- ☑ Campo "Correo institucional UAT" (solo `@uat.edu.mx`)
- ☑ Campo "Contraseña" con requisitos
- ☑ Campo "Confirmar contraseña"
- ☑ Upload de "Ficha de pago" (PDF/Imagen)
- ☑ Checkbox "Acepto términos y condiciones"
- ☑ Botón "Crear Cuenta"
- ⏳ Validación de matrícula contra base de datos UAT
- ⏳ Verificación por email

Proceso de Registro:

1. Usuario completa formulario
2. Sube ficha de pago como comprobante
3. Sistema valida correo institucional
4. Administrador revisa y aprueba cuenta
5. Usuario recibe notificación de aprobación

Estado de Implementación:

- ☑ UI completa
- ☑ Navegación a pantalla "Pendiente de Aprobación"
- ⏳ Upload de archivos (requiere backend)
- ⏳ Sistema de aprobación (requiere backend + admin panel)

---

4 Pantalla de Validación Pendiente

Archivo: `PendingScreen.kt` / `pending.html`

Descripción: Pantalla intermedia que se muestra después del registro mientras la cuenta es validada por administradores.

Elementos Visuales:

- Icono de reloj/espera
- Mensaje de estado claro
- Instrucciones para el usuario
- Diseño minimalista y tranquilizador

Funcionalidades:

- ☑ Mensaje "Tu cuenta está en revisión"
- ☑ Información sobre el proceso de validación
- ☑ Tiempo estimado de aprobación (24-48 horas)
- ☑ Botón "Cerrar sesión"
- ⏳ Notificación push cuando se apruebe cuenta
- ⏳ Botón "Verificar estado" para consultar

Proceso de Validación:

1. Administrador revisa ficha de pago
2. Verifica que matrícula sea válida
3. Aprueba o rechaza cuenta
4. Usuario recibe email/notificación
5. Si aprueba: usuario puede acceder
6. Si rechaza: usuario recibe razón y puede re-aplicar

Estado de Implementación:

- ☑ UI completa
- ⏳ Sistema de notificaciones (pendiente)
- ⏳ Panel administrativo (pendiente)

5️⃣ Pantalla Principal (Home/Dashboard)

Archivo: `HomeScreen.kt` / `home.html`

Descripción: Hub central de la aplicación después del login. Muestra estadísticas del usuario y acceso a todos los módulos.

Elementos Visuales:

- Top App Bar con foto de perfil y notificaciones
- Sección de estadísticas personales (2 cards)
- Grid de módulos con iconos
- Bottom Navigation Bar
- Diseño tipo dashboard moderno

Funcionalidades:

Top Bar:

- ☑ Foto de perfil (clickeable → Perfil)
- ☑ Título "DevCore"
- ☑ Icono de notificaciones con badge

Estadísticas:

- ☑ Card "Reputación" - Muestra nivel de confianza (0-100)
- ☑ Card "Racha" - Días consecutivos usando la app
- 📊 Valores de ejemplo: Reputación 85, Racha 7 días

Módulos Principales:

- ☑ 🚌 **Seguimiento de Bus** - Monitoreo en tiempo real
- ☑ 🛒 **Marketplace** - Compra/venta entre estudiantes
- ☑ 👤 **Perfil** - Configuración y datos personales
- ⏳ 📚 Biblioteca Virtual (planificado)
- ⏳ 🎓 Horario de Clases (planificado)
- ⏳ 💬 Foro Estudiantil (planificado)

Bottom Navigation:

- ☑ Inicio
- ☑ Bus
- ☑ Market
- ☑ Perfil

Estado de Implementación:

- ☑ UI completa con todas las secciones
- ☑ Navegación entre módulos funcional
- ⏳ Datos reales desde backend (mostrando placeholders)
- ⏳ Notificaciones push

---

6 Pantalla de Seguimiento de Bus

Archivo: `BusTrackingScreen.kt` / `bus-tracking.html`

Descripción: Módulo de monitoreo en tiempo real del transporte universitario. Permite ver ubicación de buses y reportar problemas.

Elementos Visuales:

- Mapa interactivo (placeholder)
- Card de información del bus
- Botones de reporte
- Lista de buses disponibles
- Bottom Navigation

Funcionalidades:

Monitoreo en Tiempo Real:

- ⏳ Mapa con ubicación GPS de buses
- ⏳ Ruta del bus en el mapa
- ⏳ Tiempo estimado de llegada a parada
- ⏳ Indicador de ocupación del bus (vacío/medio/lleno)

Sistema de Reportes:

- ☑ Botón "Reportar Problema"
- ☑ Botón "Confirmar Ubicación"
- ⏳ Categorías de reporte:
  - Bus no llegó
  - Bus lleno
  - Desvío de ruta
  - Problema mecánico
  - Conductor
- ⏳ Sistema de karma: reportes correctos aumentan reputación

Información Mostrada:

- Nombre/número de ruta
- Hora de última actualización
- Próximas paradas
- Capacidad actual

Gamificación:

- ⏳ +5 reputación por reporte verificado
- ⏳ -10 reputación por reporte falso
- ⏳ Badge "Guardian del Transporte" por 50 reportes correctos

Estado de Implementación:

- ☑ UI base con placeholder de mapa
- ☑ Botones de reporte
- ⏳ Integración con Google Maps API
- ⏳ Sistema GPS en tiempo real (requiere backend + hardware en buses)
- ⏳ Sistema de verificación de reportes

---

7 Pantalla de Perfil

Archivo: `ProfileScreen.kt` / `profile.html`

Descripción: Perfil personal del usuario con estadísticas, configuración y logros.

Elementos Visuales:

- Header con foto de perfil grande
- Nombre y carrera del usuario

- Sección de confianza con barra de progreso
- Estadísticas de actividad
- Grid de logros/badges
- Botón de cerrar sesión

Funcionalidades:

Información Personal:

- ☑ Foto de perfil (editable)
- ☑ Nombre completo
- ☑ Carrera/Facultad
- ⏳ Matrícula
- ⏳ Semestre actual
- ⏳ Botón "Editar perfil"

Sistema de Confianza:

- ☑ Indicador visual "Nivel de Confianza"
- ☑ Barra de progreso (0-100%)
- 📊 Valor actual: 85/100
- ⏳ Desglose de cómo se calcula:
  - Verificación de identidad: +20
  - Tiempo en plataforma: +10
  - Transacciones exitosas: +5 cada una
  - Reportes verificados: +5 cada uno
  - Calificaciones positivas: +3 cada una

Estadísticas de Actividad:

- ☑ Racha actual (días consecutivos): 7 días
- ☑ Publicaciones en marketplace: 3
- ⏳ Transacciones completadas
- ⏳ Reportes de bus enviados
- ⏳ Calificación promedio recibida

Sistema de Logros:

- ☑ Grid de badges/medallas
- Logros planificados:
  - 🔥 "Racha de Fuego" - 30 días consecutivos
  - 🚌 "Guardian del Bus" - 50 reportes correctos
  - 🛒 "Vendedor Estrella" - 20 ventas exitosas
  - ⭐ "Confianza Total" - 100 de reputación
  - 🎓 "Veterano" - 1 año en la plataforma
  - 💎 "Elite UAT" - Todos los logros desbloqueados

Configuración:

- ☑ Botón "Cerrar Sesión"

- ⏳ Cambiar contraseña
- ⏳ Notificaciones
- ⏳ Privacidad
- ⏳ Eliminar cuenta

Estado de Implementación:

- ☑ UI completa con todas las secciones
- ☑ Valores de ejemplo funcionando
- ⏳ Edición de perfil
- ⏳ Upload de foto de perfil
- ⏳ Cálculo real de reputación (backend)
- ⏳ Sistema de logros desbloqueable

---

8 Pantalla de Marketplace

Archivo: `MarketplaceScreen.kt` / `marketplace.html`

Descripción: Tienda/marketplace para compra-venta entre estudiantes de la UAT. Productos escolares, libros, apuntes, etc.

Elementos Visuales:

- Top App Bar con búsqueda
- Botón flotante "Publicar Producto"
- Grid de productos con cards
- Cada card muestra: imagen, título, precio, vendedor
- Bottom Navigation

Funcionalidades:

Exploración de Productos:

- ☑ Grid responsivo de productos
- ☑ Cada card muestra:
  - Imagen del producto
  - Título
  - Precio
  - Nombre del vendedor
  - Indicador de confianza del vendedor
- ⏳ Búsqueda por texto
- ⏳ Filtros por:
  - Categoría (Libros, Apuntes, Calculadoras, Electrónicos, Ropa, Accesorios)
  - Rango de precio
  - Facultad
  - Estado del producto (Nuevo/Usado)
- ⏳ Ordenar por:
  - Más reciente
  - Precio: menor a mayor

    - Precio: mayor a menor
    - Mejor calificado

Publicar Producto:

- ☑ Botón flotante "+"
- ⏳ Formulario de publicación:
    - Fotos del producto (hasta 5)
    - Título
    - Descripción
    - Precio
    - Categoría
    - Estado (Nuevo/Usado)
    - Ubicación (Facultad/Campus)
    - Método de entrega (Presencial/Envío)
- ⏳ Validación: solo usuarios con reputación >30 pueden publicar

Sistema de Transacción:

- ⏳ Chat entre comprador y vendedor
- ⏳ Negociación de precio
- ⏳ Sistema de "Apartado" (reserva temporal)
- ⏳ Calificación mutua después de transacción
- ⏳ Sistema de denuncia por fraude

Gamificación:

- ⏳ +10 reputación por primera venta
- ⏳ +5 reputación por cada venta con calificación 5 estrellas
- ⏳ -20 reputación por denuncia verificada de fraude

Productos de Ejemplo (Mostrados):

- 📊 Cálculo Diferencial - $150
- 💻 Laptop HP - $5000
- 📚 Apuntes Física - $80
- 🧮 Calculadora Científica - $300

Estado de Implementación:

- ☑ UI con grid de productos
- ☑ Cards diseñadas
- ☑ Botón flotante de publicar
- ⏳ Búsqueda y filtros
- ⏳ Sistema de publicación
- ⏳ Chat integrado
- ⏳ Sistema de transacciones (backend)
- ⏳ Calificaciones

📊 Estado General del Proyecto

☑ Completado (100%)

Prototipo HTML/CSS

- ☑ 8 pantallas completas
- ☑ Diseño responsivo
- ☑ Paleta de colores UAT pastel
- ☑ Navegación entre pantallas
- ☑ Estilos consistentes

Aplicación Kotlin/Compose

- ☑ 8 pantallas Composables
- ☑ Sistema de navegación completo
- ☑ Tema personalizado UAT
- ☑ Colores, tipografía y componentes
- ☑ Configuración de Gradle
- ☑ AndroidManifest
- ☑ MainActivity

⏳ En Progreso (0%)

Backend Python

- ⏳ No iniciado
- Planificado: Flask/FastAPI
- Base de datos: PostgreSQL

Funcionalidades Core

- ⏳ Autenticación JWT
- ⏳ Sistema de usuarios
- ⏳ APIs REST
- ⏳ Sistema de archivos (upload)

📅 Planificado

Integraciones

- 🗺 Google Maps API (Bus Tracking)
- 📧 Servicio de email (verificación)
- 🔔 Notificaciones push
- 📱 Integración con hardware GPS en buses

Funcionalidades Avanzadas

- 💬 Chat en tiempo real (WebSockets)

- 📊 Panel administrativo web
- 📈 Analytics y reportes
- 🔒 Sistema de moderación
- ⚖️ Sistema de resolución de disputas

🏗️ Arquitectura Técnica

Frontend Android (Kotlin)

```
DevCore/
├── ui/
│   ├── theme/
│   │   ├── Color.kt          - Paleta UAT
│   │   ├── Theme.kt          - MaterialTheme
│   │   └── Type.kt           - Tipografía
│   └── screens/
│       ├── WelcomeScreen.kt
│       ├── LoginScreen.kt
│       ├── RegisterScreen.kt
│       ├── PendingScreen.kt
│       ├── HomeScreen.kt
│       ├── BusTrackingScreen.kt
│       ├── ProfileScreen.kt
│       └── MarketplaceScreen.kt
├── navigation/
│   ├── Screen.kt             - Rutas
│   └── NavGraph.kt           - NavHost
└── MainActivity.kt           - Entry point
```

Backend (Planificado)

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py              - FastAPI app
│   ├── auth/
│   │   ├── routes.py        - Login, registro
│   │   └── jwt.py           - Token management
│   ├── users/
│   │   ├── models.py
│   │   └── routes.py
│   ├── marketplace/
│   │   ├── models.py
│   │   └── routes.py
│   ├── bus_tracking/
│   │   ├── models.py
│   │   └── routes.py
│   └── database.py          - SQLAlchemy config
```

```
  ├── requirements.txt
  └── .env
```

📈 Progreso del Proyecto

Sprint 1 - Diseño Visual ✅ (100%)

- ✅ Prototipo HTML completo
- ✅ Paleta de colores definida
- ✅ 8 pantallas diseñadas
- ✅ Navegación funcional

Sprint 2 - Conversión a Kotlin ✅ (100%)

- ✅ Migración a Jetpack Compose
- ✅ Sistema de navegación
- ✅ Tema personalizado
- ✅ Todas las pantallas funcionales

Sprint 3 - Backend (Próximo) ⏳ (0%)

- ⏳ Configuración FastAPI
- ⏳ Base de datos PostgreSQL
- ⏳ Autenticación JWT
- ⏳ APIs de usuarios

Sprint 4 - Integración ⏳ (0%)

- ⏳ Conectar frontend con backend
- ⏳ ViewModels y estados
- ⏳ Llamadas HTTP con Retrofit
- ⏳ Manejo de errores

Sprint 5 - Funcionalidades Avanzadas ⏳ (0%)

- ⏳ Google Maps integración
- ⏳ Chat en tiempo real
- ⏳ Sistema de notificaciones
- ⏳ Upload de archivos

🎯 Métricas del Proyecto

Líneas de Código

- **Kotlin:** ~1,200 líneas
- **HTML/CSS:** ~800 líneas
- **Total:** ~2,000 líneas

Archivos Creados

- **Kotlin:** 15 archivos
- **HTML:** 8 archivos
- **CSS:** 1 archivo
- **Configuración:** 4 archivos
- **Total:** 28 archivos

Pantallas Implementadas

- **Total:** 8 pantallas
- **Funcionales:** 8/8 (100%)
- **Con backend:** 0/8 (0%)

🚀 Próximos Pasos

Corto Plazo (1-2 semanas)

1. ☑ Verificar compilación en Android Studio
2. ⏳ Implementar backend básico en Python
3. ⏳ Crear endpoints de autenticación
4. ⏳ Conectar login/registro con backend

Mediano Plazo (1 mes)

1. ⏳ Implementar base de datos completa
2. ⏳ Sistema de upload de archivos
3. ⏳ Panel administrativo de aprobación
4. ⏳ Marketplace funcional con CRUD

Largo Plazo (2-3 meses)

1. ⏳ Integración Google Maps
2. ⏳ Sistema GPS en buses
3. ⏳ Chat en tiempo real
4. ⏳ Notificaciones push
5. ⏳ Deploy en producción

👥 Roles y Responsabilidades

Desarrollador Frontend

- ☑ Diseño de UI/UX
- ☑ Implementación en Kotlin/Compose
- ⏳ Integración con APIs

Desarrollador Backend

- ⏳ APIs REST
- ⏳ Base de datos
- ⏳ Lógica de negocio
- ⏳ Autenticación

DevOps

- ⏳ Configuración de servidores
- ⏳ CI/CD
- ⏳ Monitoreo

Administrador

- ⏳ Aprobación de cuentas
- ⏳ Moderación de contenido
- ⏳ Resolución de disputas

📝 Notas Técnicas

Decisiones de Diseño

¿Por qué Jetpack Compose?

- Moderno y recomendado por Google
- Menos código que XML
- UI declarativa
- Mejor performance

¿Por qué FastAPI para backend?

- Rápido y eficiente
- Documentación automática (Swagger)
- Type hints de Python
- Async/await nativo

¿Por qué sistema de reputación?

- Evitar fraudes en marketplace
- Incentivar reportes correctos en bus tracking
- Crear comunidad confiable
- Gamificación aumenta engagement

Desafíos Identificados

1. **Validación de estudiantes:** Requiere integración con sistema UAT
2. **GPS en buses:** Necesita hardware y coordinación con universidad
3. **Escalabilidad:** Preparar para miles de usuarios
4. **Moderación:** Sistema para detectar contenido inapropiado
5. **Privacidad:** Proteger datos personales de estudiantes

Información de Contacto

Proyecto: DevCore UAT
Desarrollador: [Nombre]
Email: [email]
GitHub: [repositorio]

Licencia

Este proyecto es propiedad de [Tu nombre/organización].
Todos los derechos reservados © 2026

Generado: Marzo 2026
Versión del documento: 1.0